NTNU

Norwegian University of
Science and Technology

Practical Anonymous Communication with Homomorphic Encryption

Angelo, Imre

Submission date: July 2026
Main supervisor: Associate Professor Park, Jeongeun, NTNU

Norwegian University of Science and Technology
Department of Information Security and Communication Technology

Problem description

Many anonymous communication systems have been proposed and implemented with the goal of allowing users to exchange messages over a network without revealing who is communicating with whom. Most of these designs rely on the *threshold model* to provide anonymity, wherein some components of their infrastructure must be behaving honestly. Systems with stronger anonymity guarantees generally suffer from poor scalability or are impractical to instantiate.

This thesis examines a new construction called a (Somewhat) Practical Anonymous Router (sPAR), which addresses the trust and practicality concerns of existing systems. The construction relies only on well-established Fully Homomorphic Encryption (FHE) schemes, making it realistically implementable. While sPAR presents a promising new avenue for constructing anonymous communication systems, it remains a theoretical construction. A concrete implementation and a deeper investigation is therefore needed to evaluate its practical performance and viability in larger networks.

In an effort to investigate sPAR's viability, this thesis aims to implement the scheme and conduct a structured evaluation of its performance with respect to the number of participants in the system.

Research Questions:

- What is the computational cost of sPAR, and how does it compare to the theoretical complexity described in the original paper?
- How does the performance of sPAR scale with the number of participants, and at what point does it become impractical?

Approved: 2026-04-01 – Associate Professor Park, Jeongeun, NTNU (Main supervisor)

Abstract

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

And after the second paragraph follows the third paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Sammendrag

After this fourth paragraph, we start a new paragraph sequence. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

This is the second paragraph. Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Preface

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1
Background

Anonymous communication aims to enable users to exchange messages over a network without revealing who is communicating with whom, thereby protecting both sender and receiver identities. Many such systems have been proposed and implemented over the years, such as Tor, I2P, and mixnets. Most of these designs depend on the *threshold model* to provide anonymity, requiring at least some components of their infrastructure to behave honestly. In practice, this assumption is increasingly strained by large-scale surveillance, correlation attacks, and the growing centralization of internet infrastructure [SEV+15].

Systems with stronger anonymity guarantees based on Multi-Party Communication (MPC) exist. However, these suffer from poor scalability due to their high interactivity, making them impractical for large-scale networks. A Dining Cryptographers network (DC-net), for example, can provide anonymity even if there are no trusted components in the network infrastructure. Doing so requires every participant to interact with every other participant, for every round of messages sent.

Recently, a new line of research has proposed fundamentally different designs, aiming to offer strong anonymity even in the presence of compromised or adversarial components in the network actively attempting to break anonymity. The first such design was the Non-Interactive Anonymous Router (NIAR) [SW21]. This paper proposed using Multi-Client Functional Encryption (MCFE) to allow a single, untrusted router to shuffle or permute messages from a set of clients *obliviously*; without learning anything about the permutation. Thereby making it infeasible for the router to link any of the messages to its sender.

NIAR was a breakthrough in anonymous communication, not only showing the theoretical possibility of designing anonymous routers, but also that such routers can be *non-interactive*: clients only send one message to the router and do not need to perform any additional interaction to achieve anonymity. Using a centralized (anonymous) router in this way yields similar benefits to existing MPC-based systems,

without needing any interactions between clients. However, there are a number of problems that makes NIAR impractical, if not impossible, to implement in practice.

1. The cryptographic components required by NIAR are extremely computationally expensive or not known to be implementable in practice.
2. Achieving anonymity for recipients assumes the existence of a very strong primitive called *indistinguishability obfuscation with sub-exponential security*. The practicality of such a construction is unknown.
3. Reusing the same setup across multiple rounds makes messages from the same sender linkable, unless the expensive setup phase is rerun for every round

- Anonymous communication
- Current solutions
- Problems with current solutions
- FHE as a new approach
- sPAR
- RSA is multiplicatively homomorphic, but (follow-up) paper theorized a *fully* homomorphic scheme may be possible [Gen09; RAD78]

Chapter 2
Related Works

This section details the fundamental building blocks of the sPAR protocol, and the protocol itself. To achieve a reasonable depth the protocol utilizes the *external product*, a primitive from TFHE bootstrapping that allows homomorphic multiplication of binary digits with near-constant noise bounds — enabling a large number of binary multiplication without consuming a level/bootstrapping. This primitive was ported to BGV-RNS for this thesis, which does not natively allow digit decomposition as normally used by the external product.

2.1 Fully Homomorphic Encryption

Homomorphic Encryption (HE) allows computation directly on encrypted data: an untrusted party can evaluate a function over ciphertexts without learning anything about the underlying plaintexts. Schemes are classified by the computations they support — Somewhat Homomorphic Encryption (SHE) and *leveled* schemes evaluate circuits of bounded (multiplicative) depth, while FHE supports circuits of arbitrary depth, typically by resetting the accumulated noise through bootstrapping.

Modern FHE schemes operate on polynomials in the ring $R_q = \mathbb{Z}_q[X]/(X^N + 1)$ and base their security on the hardness of the (decision) Ring Learning With Errors (RLWE) problem: distinguishing samples of the form $(a, a \cdot s + \epsilon)$ from uniformly random pairs in R_q^2. This section focuses on the BGV scheme [BGV11] used by the implementation of this thesis; the required TFHE primitives are presented in a later section.

Definition 2.1. RLWE ciphertext $\overrightarrow{\mathbf{x}} \in R_q^2$ containing a uniformly random *public element* a and the masked element $b = a \cdot s + \Delta m + \epsilon$ derived from a persistent secret key s and randomly sampled noise ϵ, where Δ is the message scaling factor

and $m \in R_t$ is the plaintext message being encrypted.

$$\mathsf{RLWE}(m) = (b, a) = (a \cdot s + \Delta m + \epsilon, a) \tag{2.1}$$

$$s \xleftarrow{\$} \{-1, 0, 1\}^N, \quad \epsilon \xleftarrow{\$} \chi, \quad a \xleftarrow{\$} R_q \tag{2.2}$$

An RLWE ciphertext $\overrightarrow{\mathbf{x}}$ encrypting a message m under the secret key s is denoted $\mathsf{RLWE}_s(m)$, but if the secret key is understood from context then $\mathsf{RLWE}(m)$ is typically used.

Most HE schemes support homomorphic addition and multiplication, and admit one of the two for *free*, in the sense that applying the operation directly to the ciphertexts applies it to the underlying plaintexts, without any additional machinery. For example, RSA has free multiplication, while schemes based on RLWE have free addition.

Definition 2.2. RLWE addition.

$$\overrightarrow{\mathbf{x}} + \overrightarrow{\mathbf{y}} = (b_x, a_x) + (b_y, a_y) = \mathsf{RLWE}_s(m_x + m_y) \tag{2.3}$$

Proof.

$$(a_x s + \Delta m_x + \epsilon_x, a_x) + (a_y s + \Delta m_y + \epsilon_y, a_y)$$
$$((a_x + a_y)s + \Delta(m_x + m_y) + (\epsilon_x + \epsilon_y), a_x + a_y)$$
$$\text{Let } a = (a_x + a_y) \text{ and } \epsilon = (\epsilon_x + \epsilon_y)$$
$$\therefore \overrightarrow{\mathbf{x}} + \overrightarrow{\mathbf{y}} = (as + \Delta(m_x + m_y) + \epsilon, a) = \mathsf{RLWE}_s(m_x + m_y)$$

□

Homomorphic multiplication in BGV is considerably more complicated and expensive: the product of two ciphertexts has three elements and multiplied noise terms, and requires *relinearization* — a form of key switching — to return to two elements and manage the noise growth without bootstrapping. As sPAR performs no such multiplications, the procedure is not detailed further.

Definition 2.3. Modulus Switching The process of converting a ciphertext $\overrightarrow{\mathbf{x}} \in R_q^2$ into a ciphertext $\overrightarrow{\mathbf{x}}' \in R_{q'}^2$ encrypting the same message under a smaller modulus $q' < q$, by rescaling and rounding

$$\overrightarrow{\mathbf{x}}' = \left\lceil \frac{q'}{q} \overrightarrow{\mathbf{x}} \right\rfloor, \tag{2.4}$$

where the rounding is chosen such that $\overrightarrow{\mathbf{x}}' \equiv \overrightarrow{\mathbf{x}} \pmod{t}$ to preserve correct decryption. The noise is scaled by the same factor q'/q (plus a small rounding term), so

modulus switching is the primary tool for noise management in leveled schemes. Its realization in RNS is different and will be revisited in subsection 2.2.2.

Definition 2.4. Bootstrapping The process of homomorphically evaluating the decryption circuit to reset the noise of a ciphertext, allowing further computation. Bootstrapping is what turns a leveled scheme into a *fully* homomorphic one.

Definition 2.5. Keyswitching The process of converting a message to being encrypted under a different secret key.

2.1.1 BGV

The implementation of this thesis is in the BGV cryptosystem, which places the message in the *least* significant bits of the ciphertext: $\Delta = 1$ and the noise is scaled by the plaintext modulus, $\epsilon = t \cdot \epsilon_{\mathsf{rlwe}}$, giving $b = a \cdot s + m + t\epsilon_{\mathsf{rlwe}}$. Decryption computes $m = [[b - a \cdot s]_q]_t$, which is correct as long as the noise does not wrap around q.

2.2 Residue Number Systems

In BGV, BFV and CKKS the ciphertext modulus Q is large — much larger than a computer word (typically 64 bits on modern computers). Multi-precision arithmetic is known to be slow due to its serial nature, so implementations avoid it by using a Residue Number System (RNS), which represents large numbers as a set of smaller, independent residues. RNS variants of BGV, BFV and CKKS are the standard in practice; OpenFHE, for example, only supports the RNS versions of these schemes.

Definition 2.6. Canonical Form We are interested in polynomials in the ring $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$ with degree (up to) $N - 1$ and coefficients in $[-Q/2, Q/2)$ (centered modulo representation), and different ways to represent this polynomial. The canonical form (2.5) of such a polynomial a is the form everyone is familiar with.

$$a = c_0 + c_1 X + \cdots + c_{N-1} X^{N-1} = \sum_{i=0}^{N-1} c_i X^i \tag{2.5}$$

Definition 2.7. Coefficient Form As polynomials have a well-known/predictable structure, we can omit the redundant X variable and represent $a \in R_Q$ uniquely by its coefficients. I will refer to (only) this representation (2.6) as the **coefficient** form.

$$a = (c_0, c_1, \ldots, c_{N-1}) \tag{2.6}$$

Let $Q = q_0 \ldots q_{k-1}$ with $\gcd(q_i, q_j) = 1 \; \forall \; i \neq j$, so that the Chinese Remainder Theorem (CRT) allows decomposition into smaller residues. If $\max_i\{q_i\}$ fits in a computer word then we can use CRT on the coefficients of a polynomial to work exclusively with integers that fit inside a computer word.

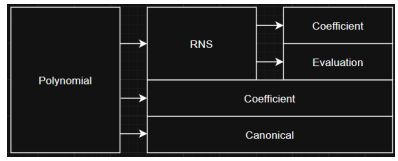
Definition 2.8. CRT Form Given a polynomial $a \in R_Q$ where $Q = q_0 q_1 \ldots q_{k-1}$ is chosen as a product of k small primes (small meaning they each fit inside a computer register), each coefficient c_i is uniquely represented by its k residues via the Chinese Remainder Theorem. Then the **CRT form** of a (2.7) is a collection of smaller polynomials $(\mathrm{mod} q_i)$ called *towers*, *residues* or *limbs*.

$$\begin{pmatrix} a \mod q_0 \\ a \mod q_1 \\ \vdots \\ a \mod q_{k-1} \end{pmatrix} = \begin{pmatrix} c_0 \bmod q_0 & c_1 \bmod q_0 & \ldots & c_{N-1} \bmod q_0 \\ c_0 \bmod q_1 & c_1 \bmod q_1 & \ldots & c_{N-1} \bmod q_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod q_{k-1} & c_1 \bmod q_{k-1} & \ldots & c_{N-1} \bmod q_{k-1} \end{pmatrix} \tag{2.7}$$

Addition and subtraction of two polynomials in R_Q is equivalent to element-wise addition/subtraction of their CRT forms (modulo q_i). As each element fits within a

computer word, no carries propagate between them, and all pairs of elements can be added or subtracted in parallel.

Remark 2.9. Rather confusingly, OpenFHE calls this form *Coefficient* format. Both RNS forms are represented by an `DCRTPoly` object, so a polynomial in CRT form is represented by a `DCRTPoly` object in `Format::Coefficient` mode.



2.2.1 Number Theoretic Transform

The Number Theoretic Transform (NTT) converts a negacyclic convolution in the coefficient domain into point-wise multiplication in the NTT domain — and multiplication in R_{q_i} is exactly such a convolution of the coefficient vectors. Applying the NTT to each residue (row) of the CRT form (2.7) therefore makes polynomial multiplication element-wise as well. The negacyclic NTT of size N requires each modulus to satisfy $q_i \equiv 1 \pmod{2N}$, which constrains the choice of RNS primes.

Definition 2.10. Double CRT/NTT/Evaluation Form By applying the NTT to the CRT form (2.7) we gain the ability to perform polynomial multiplication in parallel, without losing the ability to perform addition/subtraction in parallel.

$$\begin{pmatrix} \mathsf{NTT}(a \bmod q_0) \\ \mathsf{NTT}(a \bmod q_1) \\ \vdots \\ \mathsf{NTT}(a \bmod q_{k-1}) \end{pmatrix} = \begin{pmatrix} \hat{a}_{0,0} & \hat{a}_{0,1} & \dots & \hat{a}_{0,N-1} \\ \hat{a}_{1,0} & \hat{a}_{1,1} & \dots & \hat{a}_{1,N-1} \\ \vdots & \vdots & \ddots & \vdots \\ \hat{a}_{k-1,0} & \hat{a}_{k-1,1} & \dots & \hat{a}_{k-1,N-1} \end{pmatrix} \tag{2.8}$$

The evaluation form (2.8) is the preferred form, as it supports parallel addition, subtraction and multiplication, but it still does not natively support *all* necessary operations. Because switching between canonical/vector, CRT and NTT mode is relatively expensive — requiring (inverse) CRTs and NTTs — much of the relevant literature [GHS12; BEHZ16; HPS18; KPZ21] improves upon existing functions by finding new approaches and approximations that limit the number of format-switches required.

$$\text{Coefficient} \underset{\text{iCRT}}{\overset{\text{CRT}}{\rightleftarrows}} \text{CRT} \underset{\text{iNTT}}{\overset{\text{NTT}}{\rightleftarrows}} \text{Evaluation (DCRT)}$$

2.2.2 RNS Base Conversion

Several ciphertext maintenance operations — most importantly the key-switching procedures of the next section — temporarily extend the ciphertext modulus Q by an auxiliary modulus P and later scale back down. In RNS this corresponds to moving a polynomial between different RNS bases. I start by introducing some terminology.

Definition 2.11. RNS Basis. Given a large modulus $Q = \prod_{i=0}^{k-1} q_i$ composed of pairwise coprime factors, we define the *RNS basis* $\mathcal{B}_Q = \{q_0, q_1, \ldots, q_{k-1}\}$ as the set of RNS moduli used in the RNS/CRT representation (2.7).

Definition 2.12. Base conversion Let $\mathcal{B}_Q = \{q_0, \ldots, q_{k-1}\}$ and $\mathcal{B} = \{b_0, \ldots, b_{\ell-1}\}$ be RNS bases with $\gcd(Q, B) = 1$, where $B = \prod_j b_j$. Base conversion is the map that takes the representation of x in $\mathcal{B}_Q$ to the representation of $[x]_Q$ in $\mathcal{B}$, i.e.

$$\big([x]_{q_0}, \ldots, [x]_{q_{k-1}}\big) \longmapsto \big([x]_{b_0}, \ldots, [x]_{b_{\ell-1}}\big).$$

When the residues in $\mathcal{B}$ are appended to those in $\mathcal{B}_Q$, so that x becomes represented in the *extended* basis $\mathcal{B}_Q \cup \mathcal{B}$ for the modulus QB, the operation is also referred to as **base extension**.

Remark 2.13. Extend mod notation for RNS polynomials: $[a]_{\mathcal{B}_Q} = ([a]_{q_0}, \ldots, [a]_{q_{k-1}})$ for the basis $\mathcal{B}_Q = \{q_i\}_{i=0}^{k-1}$ means a must be in RNS form (TODO: distinguish CRT vs NTT mode... maybe $[a]_{\mathcal{B}_Q}^{\mathsf{NTT}}$?)

The naive approach to base conversion reconstructs a from its residues via the CRT and then reduces the result modulo each modulus of the new basis. This reintroduces the multi-precision arithmetic that RNS was meant to avoid. A cheap but approximate alternative exists, described next.

Fast Base Conversion

Fast base conversion, introduced by [BEHZ16], performs the conversion directly on the residues (in CRT mode), avoiding the CRT reconstruction and multi-precision integers entirely. The price is approximation: (2.9) produces not x but $x + \alpha_x Q$ for some integer $\alpha_x \in [0, k)$, as computing the exact overflow α_x would require costly operations in RNS.

$$\mathsf{FastBConv}(x, \mathcal{B}_Q, \mathcal{B}) = \left(\left[\sum_{i=0}^{k-1} \left[x_i \left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \times \frac{Q}{q_i} \right]_b \right)_{b \in \mathcal{B}} \tag{2.9}$$

Fast Base Extension

We can use (2.9) as a subroutine in base extension: only the new residues $\mathcal{B}_Q \longmapsto \mathcal{B}_P$ need to be calculated, and since $\mathcal{B}_{QP} = \mathcal{B}_Q \cup \mathcal{B}_P$ the original residues are simply kept.

Algorithm 2.1 Fast Base Extension

Input: $[a]_{\mathcal{B}_Q}$ and a basis $\mathcal{B}_P$
Output: $[a + \epsilon]_{\mathcal{B}_{QP}}$, where $\epsilon = \alpha_a Q$ for some $\alpha_a \in [0, k)$
1: $a_Q := \mathsf{InverseNTT}(a)$ ▷ Convert to CRT form
2: $a_P := \mathsf{FastBConv}(a_Q, \mathcal{B}_Q, \mathcal{B}_P)$
3: **return** $a \cup \mathsf{NTT}(a_P)$

Notably, (2.9) is approximate as a *conversion* but exact as an *extension*: the extended residue vector over $\mathcal{B}_Q \cup \mathcal{B}_P$ is an exact RNS representation of the lifted integer $x' = [x]_Q + \alpha_x Q \in \mathbb{Z}_{QP}$, since $\alpha_x Q \equiv 0 \pmod{q_i}$ keeps the original (reused) Q-residues consistent with x'. The RNS schemes are designed to tolerate this overflow, which is removed — or shrunk to the small α_x — by a subsequent division during mod down.

Remark 2.14. The factors $[(\frac{Q}{q_i})^{-1}]_{q_i}$ and $\frac{Q}{q_i}$ can be pre-calculated and stored in memory (will be discussed in **??**).

Modulus Switching

In RNS, modulus switching — also referred to as *mod down* — from Q to $Q' = Q/q_{k-1}$ amounts to *dropping the last limb* with a small correction, computed limb-wise without CRT reconstruction:

$$[c']_{q_i} = \left[q_{k-1}^{-1}\,(c - \delta)\right]_{q_i}, \quad 0 \le i < k-1, \tag{2.10}$$

where $\delta \equiv c \pmod{q_{k-1}}$ and, for BGV, $\delta \equiv 0 \pmod{t}$ so that decryption correctness is maintained. The noise shrinks by a factor q_{k-1}, matching the classical modulus switch.

Approximate Mod Down $QP \to Q$

The reverse operation, switching from the extended modulus QP back down to Q, uses (2.9) to convert the P-part back to $\mathcal{B}_Q$ before dividing by P [GHS12; HPS18]:

$$\mathsf{ModDown}(x) = \left[P^{-1}\,(x - \mathsf{FastBConv}([x]_{\mathcal{B}_P}, \mathcal{B}_P, \mathcal{B}_Q))\right]_{\mathcal{B}_Q} \tag{2.11}$$

For BGV the converted term must additionally be corrected modulo t to preserve the plaintext [KPZ21]. This operation also corrects the fast base conversion error: the conversion overflow αP is divided by P, shrinking it to the small additive term α — which is why the approximation in (2.9) is tolerable.

2.3 Noise Management Techniques in RNS

– A problem that arises in several areas of FHE is that of multiplying a ciphertext by some large number while keeping the error low.

$$(a \cdot s + \Delta m + \epsilon) \cdot X = aX \cdot s + \Delta X m + X\epsilon \tag{2.12}$$

2.3.1 Gadget Decomposition

Decomposition; a technique used to represent a number in a smaller base (radix) while preserving its value.

Definition 2.15. Digit Decomposition takes an integer $\gamma \in \mathbb{Z}_q$ (for a modulus $q \geq 2$), base β and *decomposition parameter* ℓ, and maps γ to a sum of ℓ base-β digits $\gamma_i \in [0, \beta),\ 0 \leq i < \ell$.

$$\gamma = \gamma_0 \frac{q}{\beta} + \gamma_1 \frac{q}{\beta^2} + ...\gamma_{\ell-1} \frac{q}{\beta^\ell} = \sum_{i=1}^{\ell-1} \gamma_i \frac{q}{\beta^{i+1}} \tag{2.13}$$

The *decomposition of* γ, the digits γ_i, is denoted $\mathsf{Decomp}(\gamma) = (\gamma_0, \gamma_1, \ldots, \gamma_{\ell-1})$.

– The concept of decomposition can be generalized as below
– The important property in practice is that...

Definition 2.16. Gadget Property. Two mappings $P(x), D(y)$ are said to have the gadget property iff. they satisfy (2.14) for any pair of integers $(x, y) \in \mathbb{Z}_q^2$ with some small noise $\epsilon \ll q$. If $\epsilon = 0$ then $P(x), D(y)$ are said to have the *exact* gadget property.

$$\langle P(x), D(y) \rangle = x \cdot y + \epsilon \pmod{q} \tag{2.14}$$

Definition 2.17. Gadget Decomposition generalizes the notion of digit decomposition by replacing the base β with a *gadget vector* $g = (g_0, g_1, \ldots, g_{\ell-1})$ containing ℓ elements, such that $\mathsf{Decomp}(\gamma)$ and g satisfy (2.14).

$$\gamma + \epsilon = \sum_{i=0}^{\ell-1} \gamma_i g_i \quad \iff \quad \langle \mathsf{Decomp}(\gamma), g \rangle = \gamma + \epsilon$$

– Can be applied component-wise to polynomials $\gamma \in R_q$ instead of $\mathbb{Z}_q$

The remainder of this section presents the three noise-management approaches for key switching used in practice. Following the convention of [KPZ21], each is named after the authors who introduced its core idea, though the concrete instantiations have since evolved — in particular toward RNS-friendly variants: the BV method, which controls noise purely through gadget decomposition; the GHS method, which uses no decomposition and instead controls noise through a temporary modulus extension; and the hybrid method, which combines both.

2.3.2 Brakerski-Vaikuntanathan

Brakerski and Vaikuntanathan [BGV11] introduced the idea of decomposition-based key switching: rather than multiplying a full-magnitude ciphertext element into a single key, the element is gadget-decomposed and each small digit multiplies a separate key component, so that the key noise is amplified only by digits rather than by a factor of Q. We refer to any key switching whose noise control derives entirely from this mechanism as the BV method, regardless of the particular gadget employed.

- Gadget decomposition
- Named BV after [**bv**] which first used *bit* decomposition; digit decomposition with $\beta = 2$.
- Digit decomposition is not natively compatible with RNS, at least not over the full polynomial
- First RNS instantiation using CRT decomposition [BEHZ16]
- HPS optimization for specifically keyswitching [HPS18]
- Can apply digit decomposition to each residue polynomial

2.3.3 Gentry-Halevi-Shoup

- Pre-scale the key to QP
- Extends modulus $Q \to QP$ [GHS12]
- Drops modulus back to Q after inner product, dividing by P in the process
- Since key was pre-scaled, ϵ shrinks relative to the message
- Cite to RNS section, base conversion

2.3.4 Hybrid

- BV and GHS represent a spectrum
- Combination of the two techniques

2.4 Noise Management Techniques in RNS

2.4.1 Gadget Decomposition

2.4.2 Gadget Constructions

2.4.3 Modulus Extension

2.4.4 Key-Switching Methods

2.5 TFHE Bootstrapping Primitives

- TFHE bootstrapping achieves fast evaluation of binary circuits.
- TFHE is optimized for computations of large multiplicative depth on small plaintexts, and is best known for its fast bootstrapping procedure.
- It is based on (approximating) *gadget decomposition*.
- Asymmetry in noise scaling of GSW ciphertexts; sum of small errors is smaller than product of large errors.

The rest of the lineage, as it's conventionally attributed: the ring variant (RGSW) appears in Ducas–Micciancio's FHEW (Eurocrypt 2015), which uses ring-GSW accumulators for bootstrapping, and in Khedr–Gu–Solihin's SHIELD around the same time. The external product — the asymmetric RGSW $\times$ RLWE multiplication, as opposed to GSW's original symmetric product — is due to Chillotti–Gama–Georgieva–Izabachène (TFHE, Asiacrypt 2016; journal version 2020), with a concurrent equivalent ("hybrid product") in Brakerski–Perlman (CRYPTO 2016). TFHE formulates it over the torus, i.e. TLWE/TGSW. The internal product (RGSW $\times$ RGSW) is the ring/torus version of GSW's original multiplication.

Definition 2.18. RGSW Ciphertext. etc. etc.

$$C = Z + \mu G \tag{2.15}$$

2.6 TFHE

- Attribution / lineage: GSW encryption introduced over plain LWE [**GSW13**]; ring variant (RGSW) used in FHEW [**DM15**] and SHIELD [**KGS16**]; external product and the TFHE scheme [**CGGI16**] (journal version [**CGGI20**]), concurrently [**BP16**] ("hybrid product")
- Notation remark (the torus pivot, once and for all): TFHE is natively formulated over the (discretized) torus, with TLWE/TGSW in place of RLWE/RGSW; the discretized torus $q^{-1}\mathbb{Z}/\mathbb{Z}$ is isomorphic to $\mathbb{Z}_q$ via $x \mapsto qx$ [**Joye22**], under which the torus gadget entries $1/B^j$ become q/B^j. All statements transfer verbatim; we use ring notation over R_Q throughout
- Standing assumptions: R_Q as in Sec. **??**; RLWE ciphertexts $c = (c_0, c_1) \in R_Q^2$ with phase $c_1 - s\,c_0 = \mu + e$ as in Sec. **??**; gadget pairs (D, P) and the (approximate) gadget property as in Def. **??**

2.6.1 RGSW Ciphertexts

- Fix the radix gadget (Def. **??**, Ex. **??**): $g = P_\omega(1) = (q/B, \ldots, q/B^\ell)^T$; gadget matrix
$$G = I_2 \otimes g \in R_Q^{2\ell \times 2} \tag{2.16}$$
- RGSW encryption of $\mu \in R$:
$$C = Z + \mu \cdot G \in R_Q^{2\ell \times 2}, \qquad \text{each row of } Z \text{ an RLWE encryption of } 0 \tag{2.17}$$
- Structure: the two row-blocks of C are RLWE encryptions of $\mu \cdot g_j \cdot (-s)$ and $\mu \cdot g_j$ respectively — i.e. an RGSW ciphertext is 2ℓ RLWE ciphertexts encrypting gadget-scaled copies of μ (and μs)
- RGSW is additively homomorphic; decryption via any row of the bottom block

2.6.2 External Product

- Gadget decomposition of an RLWE ciphertext, applied per component:
$$G^{-1}(c) = \big(D(c_0) \,\big|\, D(c_1)\big) \in R^{2\ell}, \qquad \text{satisfying } G^{-1}(c)\cdot G = (c_0, c_1) + (e_0, e_1), \quad \|e_i\|_\infty \le \varepsilon \tag{2.18}$$
- External product [**CGGI16**; **BP16**]:
$$C \boxdot c = G^{-1}(c) \cdot C \in R_Q^2 \tag{2.19}$$
- Correctness (one-line expansion):
$$G^{-1}(c)\cdot(Z + \mu_C G) = \underbrace{G^{-1}(c) \cdot Z}_{\text{decomposition–noise term}} + \mu_C \cdot \big(c + e_{\text{dec}}\big) \;=\; \text{RLWE}(\mu_C \cdot \mu_c) + \text{noise}$$

– Noise (statement only, proof deferred to Thm. 2.19):

$$\|v_{\boxdot}\|_\infty \lesssim \underbrace{2\ell\, n\, \beta\, B_Z}_{\langle D(c_i),\, \vec{\mathbf{e}}_Z\rangle} + \underbrace{\|\mu_C\|_1 \cdot \varepsilon}_{\mu_C \cdot e_{\mathrm{dec}}} + \|\mu_C\|_1 \cdot \|v_c\|_\infty$$

with β the digit bound of D, B_Z the RGSW noise

– Two consequences worth flagging: (i) noise is *asymmetric* — the RLWE input's noise enters scaled by μ_C, the RGSW noise only through small digits; hence μ_C must be small (typically $\mu_C \in \{0, 1\}$ or monomials X^k), while μ_c is unconstrained; (ii) cost = 2ℓ ring multiplications + the cost of D

2.6.3 Internal Product

– RGSW $\times$ RGSW $\to$ RGSW: apply the external product row-wise,

$$C_1 \boxtimes C_2 = \big(C_2 \boxdot \mathrm{row}_r(C_1)\big)_{r=1}^{2\ell} \tag{2.20}$$

each row of C_1 being an RLWE ciphertext

– Cost: 2ℓ external products; noise as in (2.19) per row

– Role: composing RGSW ciphertexts (e.g. products of controls); not used on the critical path of this thesis — included for completeness

2.6.4 Generalization

– Observation: nothing above used the radix structure of g — only the gadget property of (D_ω, P_ω)

– Constructive statement: any pair from the catalog of Ch. **??** yields a valid RGSW ciphertext and external product

Theorem 2.19. *Let $D(a), P(b) \in R_Q^\ell$ satisfy the approximate gadget property (Def. **??**) with digit bound β and decomposition error ε. Then*

$$G = I_2 \otimes P(1) \tag{2.21}$$

$$G^{-1}(x) = \big(D(x_0) \,\big|\, D(x_1)\big) \tag{2.22}$$

define a valid RGSW ciphertext (2.17) *and external product* (2.19) *with* $\mathrm{RLWE}(\mu_c) \boxdot \mathrm{RGSW}(\mu_C) = \mathrm{RLWE}(\mu_c \cdot \mu_C)$*, provided the resulting noise remains below the decryption bound.*

– Payoff / bridge: instantiating Thm. 2.19 with the gadgets of Ch. **??** gives external products computable natively in RNS — BV-type gadgets (exact, $\varepsilon = 0$) and extension-based gadgets (GHS/Hybrid, $\varepsilon > 0$ from FastBConv overflow and ApproxModDown rounding, plus the P-scaling adapted as in Sec. **??**)

– Forward remark (contribution preview, one sentence): in BGV the key/gadget ciphertexts carry the plaintext factor t (Z-rows of the form $(a, as + te)$), so the noise bounds above acquire t factors — a genuine difference from the TFHE setting, treated in Ch. **??**, not a notational one

2.7 TFHE Bootstrapping Primitives

- TFHE bootstrapping achieves fast evaluation of binary circuits.
- It is based on (approximating) *gadget decomposition*.
- Asymmetry in noise scaling of GSW ciphertexts; sum of small errors is smaller than product of large errors.

Definition 2.20. RGSW ciphertext (2.23) where Z is a vector of 2ℓ (fresh) RLWE encryptions of 0. The message $m \in \{0, 1\}$, and $G = I_2 \otimes g$ where g is the *gadget vector*, typically defined as $g = (1, 1/B, \ldots, 1/B^{\ell-1})$.

$$\mathsf{RGSW}_s(m) = Z + m \cdot G \tag{2.23}$$

The external product approximates the gadget property by defining a gadget matrix G and inverse gadget matrix $G^{-1}(x)$ s.t.

$$G^{-1}(x) \cdot G = x + \epsilon \approx x \pmod{Q} \tag{2.24}$$

Definition 2.21. External Product $\boxdot$ defines a product between an RLWE and an (Ring) Gentry-Sahai-Waters (RGSW) ciphertext. $G^{-1}(x)$ satisfies the approximate gadget property (2.24).

$$\mathsf{RLWE}_s(a) \boxdot \mathsf{RGSW}_s(b) \to \mathsf{RLWE}_s(a \cdot b) \tag{2.25}$$

$$a \boxdot B = G^{-1}(a) \cdot B \pmod{Q} \tag{2.26}$$

Definition 2.22. Internal Product $\boxtimes$ defines a product between RGSW ciphertexts by repeated external products.

$$\mathsf{RGSW}_s(a) \boxtimes \mathsf{RGSW}_s(b) = \mathsf{RGSW}_s(a \cdot b) \tag{2.27}$$

$$A \boxtimes B = \begin{bmatrix} A_0 \boxdot B \\ \vdots \\ A_{2\ell-1} \boxdot B \end{bmatrix} \tag{2.28}$$

2.7.1 Generalization

- We are not limited to the gadget vector $g = (1, 1/B, \ldots, 1/B^{\ell})$ and `Dec`
- Any choice of G and $G^{-1}(x)$ that satisfy (2.24) works
- For later it is helpful to establish constructively how any pair of vectors that satisfy (**??**) can be used to find valid gadget matrices.

Theorem 2.23. *Any pair $D(a)$, $P(b) \in \mathbb{Z}_Q^\ell$ that satisfies the approximate gadget property* (2.24) *defines an RGSW ciphertext* (2.23) *and a valid external product* (2.25) *such that* $\mathsf{RLWE}(a) \boxdot \mathsf{RGSW}(b) = \mathsf{RLWE}(a \cdot b)$ *(assuming the noise is not too large) with the (inverse) gadget matrix:*

$$G = I_2 \otimes P(1) \tag{2.29}$$
$$G^{-1}(x) = [D(x_0)|D(x_1)] \tag{2.30}$$

Proof. First, observe that the inexact gadget property holds for (2.29) and (2.30).

$$\begin{aligned} G^{-1}(x) \cdot G &= [D(x_0)|D(x_1)] \begin{pmatrix} P(1) & 0 \\ 0 & P(1) \end{pmatrix} \\ &= [D(x_0) \cdot P(1), D(x_1) \cdot P(1)] \\ &= [x_0 \cdot 1 + \epsilon_0, x_1 \cdot 1 + \epsilon_1] \\ &= [x_0, x_1] + [\epsilon_0, \epsilon_1] = x + \epsilon \pmod{Q} \end{aligned}$$

This is probably already established in [CGGI18], but is instructive for the error terms: Then see that if the inexact gadget property holds, then the external product is valid.

$$\begin{gathered} G^{-1}(x) \cdot G = x + \epsilon_G \pmod{Q} \\ x \boxdot Y = G^{-1}(x) \cdot (Z + m_y \cdot G) \\ G^{-1}(x) \cdot Z + (G^{-1}(x) \cdot G) m_y \\ G^{-1}(x) \cdot Z + (x + \epsilon_G) \cdot m_y \\ G^{-1}(x) \cdot Z + x \cdot m_y + \epsilon_G \cdot m_y \end{gathered}$$

The middle term $x \cdot m_y = (a \cdot s + m_x + \epsilon, a) \cdot m_y = ((am_y)s + (m_y m_x) + m_y\epsilon, am_y)$. By setting $a' = a \cdot m_y, \epsilon' = \epsilon \cdot m_y$ its clear that this becomes a valid RLWE encryption of $m_y \cdot m_x$ from $(a's + m_x \cdot m_y + \epsilon', a')$.

The left term $G^{-1}(x) \cdot Z$ evaluates to an RLWE encryption of 0 with some non-zero

noise. Let g_i^{-1} be the i-th element of $G^{-1}(x)$.

$$G^{-1}(x) \cdot Z = \begin{pmatrix} g_0^{-1} & \cdots & g_{2\ell-1}^{-1} \end{pmatrix} \begin{pmatrix} b_0 & a_0 \\ \vdots & \vdots \\ b_{2\ell-1} & a_{2\ell-1} \end{pmatrix}$$

$$= \begin{pmatrix} \sum_{i=0}^{2\ell-1} g_i^{-1} b_i, & \sum_{i=0}^{2\ell-1} g_i^{-1} a_i \end{pmatrix}$$

$$\text{Let } a_z = \sum_{i=0}^{2\ell-1} g_i^{-1} a_i \text{ and consider the first element}$$

$$\sum_{i=0}^{2\ell-1} g_i^{-1} b_i = \sum_{i=0}^{2\ell-1} g_i^{-1}(a_i \cdot s + \epsilon_i) = \sum_i g_i^{-1} a_i \cdot s + \sum_i g_i^{-1} e_i$$

$$s\left(\sum_i g_i^{-1} a_i\right) + \sum_i g_i^{-1} e_i = s(a_z) + (\epsilon_z) = a_z \cdot s + \epsilon_z$$

$$\implies \quad G^{-1}(x) \cdot Z = (a_z s + \epsilon_z, a_z) = \mathsf{RLWE}(0)$$

Finally, the third term simply adds some noise (assuming we are using an inexact gadget or $m = 0$).

$$\therefore \quad x \boxdot Y = \mathsf{RLWE}_s(0) + \mathsf{RLWE}_{\mathsf{s}}(\mathsf{m}_{\mathsf{x}} \cdot \mathsf{m}_{\mathsf{y}}) + (\epsilon_{\mathsf{G}} \cdot \mathsf{m}_{\mathsf{y}}) = \mathsf{RLWE}_{\mathsf{s}}(\mathsf{m}_{\mathsf{x}} \cdot \mathsf{m}_{\mathsf{y}}) \qquad \square$$

Remark 2.24. The exact gadget property is a special case of the inexact gadget property with $\epsilon = 0$, so the same proof holds.

2.7.2 Noise

Each term has a source of noise:

- $\epsilon_z = \sum_i g_i^{-1} \epsilon_i$
- $\epsilon' = m_y \cdot \epsilon$
- $\epsilon_G \cdot m_y$

Some observations:

- Two error terms are multiplied by m_y, which is why this method works best in binary systems where $m_y \in \{0, 1\}$ as this error term is bounded by the original error term $\|\epsilon'\| \leq \|\epsilon\|$
- The decomposition makes ϵ_z small because g_i^{-1} are small and a sum of many small errors is better than a product of large errors (main point of decomposition). Also, since ϵ_i are sampled from a gaussian distribution, their expected value is 0.
- The ϵ_G comes purely from the choice of gadget. If we choose an *exact* gadget with $\epsilon = 0$ then $\epsilon_G \cdot m_y = 0$ and this error vanishes.

– g_i^{-1} are also from the gadget, but is less controllable as it also depends on x. A good choice of gadget is therefore one that satisfies the exact gadget property and has small elements in $G^{-1}(x)$.

$\therefore$ The choice of gadget affects the noise and some gadgets are *better* than others, as they add less noise during the external product! Ideally we want to use a decomposition that has the *exact* gadget property, to minimize noise the noise from the *"error (left) term"*.

2.8 Gadgets in RNS (keyswitching)

Digit decomposition requires a to be in its coefficient representation (2.6), does not natively work on residues and is therefore not well suited for RNS. A few RNS-native gadgets have been proposed over the years, primarily in the context of key switching, such as [BEHZ16; HPS18] based on the BV gadget, and [GHS12] based on RNS-base extension.

2.8.1 Brakerski-Vaikuntanathan

Although we cannot use *digit* decomposition, RNS is itself based on a different decomposition: *CRT* decomposition. This means we can decompose an RNS polynomial using the RNS primes themselves as a basis as shown in [BEHZ16; KPZ21].

$$D_Q(a) = \left(\left[a \left(\frac{Q}{q_0} \right)^{-1} \right]_{q_0}, \ldots, \left[a \left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \right) \tag{2.31}$$

$$P_Q(a) = \left(\left[a \frac{Q}{q_0} \right]_Q, \ldots, \left[a \frac{Q}{q_i} \right]_Q \right) \tag{2.32}$$

Lemma 2.25. *The vectors* (C.6) *and* (C.7) *satisfy the gadget property [KPZ21].*

$$\langle D_Q(a), P_Q(b) \rangle = a \cdot b \pmod{Q} \tag{2.33}$$

It was later shown by Halevi, Polyakov and Shoup [HPS18] that the factors can be moved into $P_Q(a)$ without violating Lemma C.3. Since $[(Q/q_i)^{-1}]_{q_i}[Q/q_i] = 1$ in tower $j = i$ and 0 in towers $j \neq i$, this optimization effectively makes the decomposition free as both $D_{\mathsf{HPS}}(a) \cong a_{\mathsf{limbs}}$ and $P_{\mathsf{HPS}}(a) \cong a_{\mathsf{limbs}} \cdot I_k$ contains the same information as a, and is only structured differently. For convenience we let $\hat{q}_i = Q/q_i$ so we can write the elements of $P_{\mathsf{HPS}}(a)$ as $a[\hat{q}_i^{-1}]_{q_i}[\hat{q}_i]_Q$.

$$D_{\mathsf{HPS}}(a) = \left([a]_{q_0}, [a]_{q_1}, \cdots, [a]_{q_i} \right) \tag{2.34}$$

$$P_{\mathsf{HPS}}(a) = \left(a \cdot \left[\hat{q}_0^{-1} \right]_{q_0} \cdot [\hat{q}_0]_Q, \ldots, a \cdot \left[\hat{q}_\ell^{-1} \right]_{q_i} \cdot [\hat{q}_\ell]_Q \right) \tag{2.35}$$

- This saves many unnecessary calculations in practice (see **??**)
- The factors $[\hat{q}_i^{-1}]_{q_i}[\hat{q}_i]_Q$ are static and can be pre-calculated

2.8.2 Double Decomposition

- By Theorem 2.23, we can use these gadget vectors to define an external product.
- The noise added scales with $\sum_i q_i \leq k \cdot \max_i\{q_i\}$ (Cite error bounds from TFHE paper)
- Fine for keyswitching, but too large for some applications like internal products (2.27).
- To combat this, we can add a second layer of decomposition to each decomposed element using the classic gadget technique with $g = (1/\beta, \ldots, 1/\beta^{\ell-1})$.
- As each element is a ring element in R_q for some small q that fits inside a computer register/word, applying this method does not incur the performance penalty of using multi-precision integers.
- This can be represented as $D_{\mathsf{BV}}(a) = P_{\mathsf{BV}}(a \cdot g^{-1}(a))$ and $D_{\mathsf{BV}}(a) = P_{\mathsf{BV}}(a \cdot g)$ where $g = ($ and $g^{-1}(a)$ satisfies $g^{-1}(a) \cdot g = x)$, but this results in a matrix $P_{\mathsf{BV}} \in R_q^{2\times k}$. To fit this within our established framework, we simply flatten $R_q^{\ell\times k} \to R_q^{1\times k\ell}$.

Corollary 2.26. Correctness of Double Decomposition *Applying digit decomposition to each element of $P_{\mathsf{HPS}}(a)$ individually yields a valid external product by Theorem 2.23. The proof changes only structurally when $P_{BV}(m)$ and $D_{BV}(n)$ are flattened.*

Proof Sketch. Let $P_{\mathsf{BV}}(a) = P_{\mathsf{HPS}}(a \cdot g)$ and $D_{BV}(a) = D_{HPS}(a \cdot g^{-1})$ where $g = (1, \beta, \ldots, \beta^{\ell-1})$ and g^{-1} is the digit extraction of n such that $g^{-1}(n) \cdot g = n$. Using the fact that $P_{HPS}(\cdot), D_{HPS}(\cdot)$ satisfies the exact gadget property (**??**), we see that the same is true for $P_{BV}(\cdot), D_{BV}(\cdot)$:

$$\langle P_{BV}(m), D_{BV}(n)\rangle = \langle P_{HPS}(m \cdot g), D_{HPS}(n \cdot g^{-1})\rangle \pmod{Q} \tag{2.36}$$

$$= (m \cdot g) \cdot (n \cdot g^{-1}) = m \cdot n \cdot (g^{-1} \cdot g) \pmod{Q} \tag{2.37}$$

$$= m \cdot n \pmod{Q} \tag{2.38}$$

$$\therefore \quad \langle P_{BV}(m), D_{BV}(n)\rangle = m \cdot n \pmod{Q} \qquad \square$$

Remark 2.27. Now ℓ covers $\max_i\{q_i\}$ and the RGSW has $k \cdot \ell$ rows. This is the same length required if ℓ covers the entire Q as we would have k less numbers but each number would be k digits longer. For example, with $\ell_{small} = 2$ and $Q = q_0q_1 \implies k = 2$ we get $2 \cdot k \cdot \ell_{small} = 2 \cdot 2 \cdot 2 = 8$ RGSW rows and the noise is equivalent to having $\ell_{big} = 4 \implies 2 \cdot \ell_{big} = 8$ RGSW rows over Q. The results is

the same, the number of rows is the same, the noise is the same, its just faster to calculate over each q_i individually.

Double decomposition

2.8.3 Gentry-Halevi-Smart

The GHS technique, first introduced by Gentry, Halevi and Smart in [GHS12], is a faster method of performing keyswitching in RNS than the previous method. Later the method was improved by the BEHZ authors [BEHZ16]. At a high level, the GHS technique involves three steps:

1. Generate a public key $\mathsf{Enc}(b \cdot P)$ in QP
2. Lift the ciphertext $\mathsf{Enc}(a)$ into a higher basis $Q \to QP$
3. Apply the gadget operation in QP to get $\mathsf{Enc}(a \cdot b \cdot P) + \epsilon \pmod{QP}$
4. Modulus switch the result $QP \to Q$ to get $\mathsf{Enc}(a \cdot b) + \epsilon/P \pmod{Q}$

The relevant result of [GHS12] for our purpose was finding a way to approximate modulus switching in the evaluation representation of a polynomial, and using this to device the GHS keyswitching technique. Later, [BEHZ16] proposed a fast approximation to the base extension/lifting, operating on the CRT representation of a polynomial. The *extra* noise from the operation is divided by P in the "mod down", so the noise added by the technique itself $\epsilon \approx |\frac{Q}{P}|$ is approximately 1 if Q and P are chosen to be the same bit-length ($\implies 1 \times \epsilon_{ks}$).

2.8.4 Hybrid

Combines the best of both worlds: (single-layer) BV/BEHZ/HPS and GHS. Tunable with d_{num} where $|P| \approx |Q|/d_{num}$. I use $d_{num} = 1$ in practice, turning the hybrid scheme into (pure) GHS.

2.9 RLWE-to-RGSW Expansion

- Use RGSW to avoid relinearization of binary numbers
- For communication overhead we want send packed RLWE ciphertexts to the server
- We wish to strike a balance between the cheap communication cost of packed RLWE and the fast multiplication of RGSW
- Solution: send packed RLWE ciphertexts and *expand* them into RGSW homomorphically on the server

2.9.1 ORAM

Solution from [CCR19]. **HomExpand:** Expands an RLWE ciphertext into an RGSW ciphertext.

This expansion is described in Algorithm 4 of [CCR19], which builds on Algorithms 3 and 8 from the same paper. The algorithms are listed below (in slightly different notation) as Algorithm 2.2, Algorithm 2.3, and **??**, respectively.

Algorithm 2.2 HomExpand

Input: $\mathbf{c}_k = \text{RLWE}(\sum_{i=0}^{n-1} \frac{b_i}{nB^{k+1}}),\ b_i \in \{0,1\},\ 0 \leq k < \ell$
Input: $A = \text{RGSW}(-s)$
Output: $\mathbf{C}_i = \text{RGSW}(b_i),\ 0 \leq i < n$
for each k **do**
    $\mathbf{c'}_k := \mathsf{expandRlwe}(c_k)$
end for
for $i = 0,\ i < n$ **do**
    Initialize $\mathbf{C}_i$ as an empty $2\ell \times 2$ matrix of polynomials
    for $k = 0; k < \ell$ **do**
        $\mathbf{C}_i[k] = A \boxdot \mathbf{c'}_k[i]$
        $\mathbf{C}_i[k+\ell] = \mathbf{c'}_k[i]$
    end for
end for
return $\{\mathbf{C}_j\}_{j=0}^{n-1}$

Algorithm 2.2 takes as input ℓ RLWE ciphertexts, which in sPAR correspond to the ℓ encrypted bits of the index a_j. It also takes an RGSW ciphertext of the encrypted secret key

It begins by running a subroutine expandRlwe (Algorithm 2.3) which converts each of the ℓ RLWE ciphertexts into a matrix, like a *partial* RGSW. Then these partial RGSW's are combined into a single (complete) RGSW.

Algorithm 2.3 expandRlwe

Input: $\mathbf{c} = \text{RLWE}(\sum_{i=0}^{n-1} b_i X^i),\ b_i \in \{0,1\}$
Output: $\mathbf{c}_i = \text{RLWE}(n \cdot b_i),\ 0 \leq i < n$
$\mathbf{c}_0 := \mathbf{c}$
for $i = 0,\ i \leq \log n$ **do**
    $k = n/2^{i-1} + 1$
    for $b = 0; b < 2^i$ **do**
        $\mathbf{c}_{2b} = c_b + \text{Subs}(c_b, k)$
        $\mathbf{c}_{2b+1} = c_b - \text{Subs}(c_b, k)$
    end for
end for
return $\{\mathbf{c}_j\}_{j=0}^{n-1}$

This subroutine converts an RLWE ciphertext with ℓ slots into ℓ ciphertexts, each encoding a unique slot of the input. Subs is a function that... (EvalAutomorphism!)

Their method is fast for TFHE, but for our use-case, where $n << N$, it's faster to use the method described in [HS18] to repeatedly "rotate" the ciphertext ℓ times as in Algorithm 2.4. The FastRotate operation is supported by the OpenFHE as EvalFastRotate.

Note that the components being scaled by n is a side-effect of their implementation, and not required when using FastRotate, so the output is a list of $c_i = \mathsf{RLWE}(b_i)$.

Algorithm 2.4 expandRlwe

Input: $\mathbf{c} = \text{RLWE}(\sum_{i=0}^{n-1} b_i X^i),\ b_i \in \{0,1\}$
Output: $\mathbf{c}_i = \text{RLWE}(b_i),\ 0 \leq i < n$
$\mathbf{c}_0 := \mathbf{c}$
for $i = 0,\ i \leq \log n$ **do**
    do something...
end for
return $\{\mathbf{c}_j\}_{j=0}^{n-1}$

2.10 (Somewhat) Practical Anonymous Router

The main protocol.

2.10.1 Server Write

Algorithm 2.5 Homomorphic placing HomPlacing

```
1: let notHasWritten = RGSW*(1)
2: for d ← {0, 1, 2} do
3:     for k ← {0, 1, 2} do
4:         for i ← {0, ..., η − 1} do
5:             let h = z_{i,d} ⊠ I_{i,k} ⊠ notHasWritten
6:             L_{i,k} := L_{i,k} + h · V_r
7:             I_{i,k} := I_{i,k} − h
8:             notHasWritten := notHasWritten − h
9:         end for
10:    end for
11: end for
12: return notHasWritten
```

* The server has no obligation to encrypt its values $(\mathsf{notHasWritten}, L, I)$ and may in some cases even be actively trying to break privacy and thus cannot be trusted to add any noise. This improves the performance of the algorithm as noiseless "encryption" is marginally faster as it does not need to perform any random sampling, and makes the algorithm as a whole more tolerable to noise.

2.10.2 HomPlacing

Algorithm 1 shows the idea of Algorithm 2.

- From NIAR: sorting of random numbers = shuffling.
- Evaluate tree, each bit determines if the data is placed left or right. One level per bit means there are 2^n buckets in the end, each one representing a bit string i.e. 000, 001, 010, 011, 100, 101, 110, 111 for 3 bits.
- We can encode the bits into RGSW form.

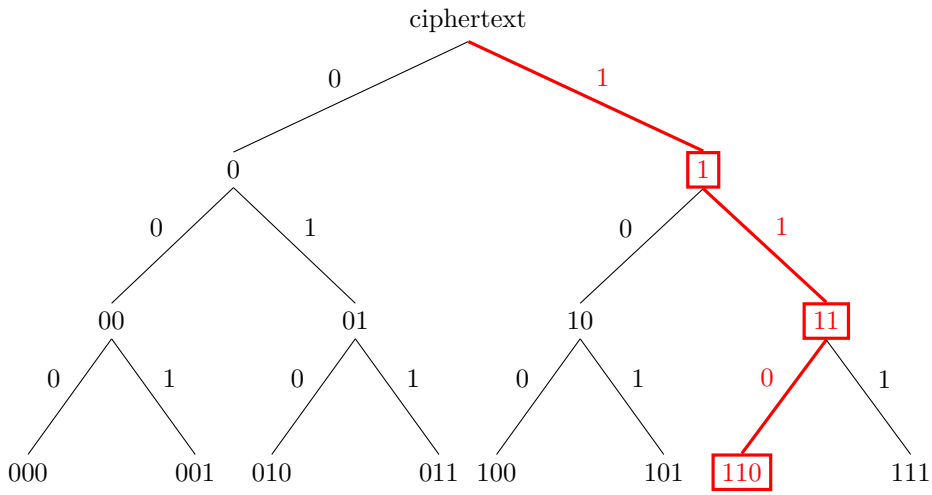


Figure 2.1: HomPlacing tree with a 3-bit index a. At level i, the i-th MSB of the index a determines homomorphically wether the ciphertext is placed in the left (0) or right (1) bucket. This process is repeated $\log n = 3$ times until the ciphertext is placed in the a-th bucket, without the placer ever knowing which bucket that is. The highlighted path shows the route taken for the index $a = 6 = [110]_2$: the MSB places the ciphertext in the right bucket, and so on. Again, the server doesn't know a, only the *encryption* of the bits $[a]_2$.

Algorithm 2.6 Homomorphic placing HomPlacing for one slot

Input: An encrypted value V_r and $\log n$ bits $\{c_0, \ldots, c_{L-1}\}$, where $L = \log n$
Output: A vector $\mathbf{l}$ consisting of n values of leaves: $z_0, \ldots, z_{n-1}$, where $\mathbf{l}[i] = z_i$ for $i \in \{0, \ldots, n-1\}$
$\mathbf{l} := \mathbf{0}$
Initialize root: $b_0 = V_r$
Initialize the node values: $b_i := 0$ for $i \in \{1, \ldots, 2n-2\}$
for $i \leftarrow 0, \ldots, L-1$ **do**
    for $j \leftarrow 0, \ldots, 2^i - 1$ **do**
        $r := b[2^i - 1 + j]$
        $b_{2^{i+1}+2j} := r \boxdot c_i$
        $b_{2^{i+1}+2j-1} := r - r \boxdot c_i$
    end for
end for
for $i \leftarrow 0, \ldots, n-1$ **do**
    $z_i := b_{2^{l+1}-1+i}$
    $\mathbf{l}[i] = z_i$
end for
return $\mathbf{l}$

Chapter 3
Methodology

3.1 Code/Library

- A result of this thesis: a C++ library which extends OpenFHE's BGV implementation to support RGSW operations
- Another result: sPAR (which was built on top of this library)
- The code base is split into three sections found in the `lib` directory:

  1. `core` extends OpenFHE's `CryptoContext<T>` with the RGSW-like operations:
     - `EncryptRGSW` outputs the RGSW encryption of the input plaintext
     - `EvalExternalProduct` takes an RLWE and RGSW ciphertext
     - `EvalInternalProduct` takes two RGSW ciphertexts
  2. `server` builds the sPAR server functions on top of `core`
  3. `client` builds the sPAR client functions on top of `core`

- There are also two auxiliary directories `benchmarks` and `tests` that contain code to benchmark and unit test the functions, respectively.

3.2 Implementation Details (RGSW/Library)

- Provides an abstract interface called `ExtendedCryptoContext` which exposes RGSW functions (EncryptRGSW, EvalExternalProduct, EvalInternalProduct etc.)
- Two concrete implementations of the `ExtendedCryptoContext` interface based on:
  - the BV (double decomposition) gadget
  - the Hybrid approach (GHS in practice)

3.2.1 BV (Double Decomposition)

Power of Base

- As remarked in **??**, the HPS gadget is equivalent to limb look-up on RNS polynomials: $P_{\mathsf{HPS}}(a) \cong a \cdot I_k$
- **??** also shows we can use the standard BV gadget on each limb of an RNS polynomial.
- The resulting gadget contains mostly 0's, and the structure is known.

$$P_{\mathsf{HPS}}(a) \cong \begin{bmatrix} [a]_{q_0} & 0 & \cdots & 0 \\ 0 & [a]_{q_1} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & [a]_{q_k} \end{bmatrix} = a \cdot I_k$$

compact storage ↙

$$\begin{bmatrix} [a]_{q_0} \\ [a]_{q_1} \\ \vdots \\ [a]_{q_k} \end{bmatrix} = \begin{bmatrix} c_0 \bmod p_0 & c_1 \bmod p_0 & \dots & c_{N-1} \bmod p_0 \\ c_0 \bmod p_1 & c_1 \bmod p_1 & \dots & c_{N-1} \bmod p_1 \\ \vdots & \vdots & \ddots & \vdots \\ c_0 \bmod p_k & c_1 \bmod p_k & \dots & c_{N-1} \bmod p_k \end{bmatrix}$$

decompose ↓

$$P_{\mathsf{BV}}(a) \cong \begin{bmatrix} [\hat{a}]_{q_0} \\ [\hat{a}]_{q_1} \\ \vdots \\ [\hat{a}]_{q_k} \end{bmatrix} = \begin{bmatrix} [a]_{q_0} & [aB]_{q_0} & \cdots & [aB^{\ell-1}]_{q_0} \\ [a]_{q_1} & [aB]_{q_1} & \cdots & [aB^{\ell-1}]_{q_1} \\ \vdots & \vdots & \ddots & \vdots \\ [a]_{q_k} & [aB]_{q_k} & \cdots & [aB^{\ell-1}]_{q_k} \end{bmatrix}$$

equivalent ↕

$$\begin{bmatrix} [a \cdot g]_{q_0} \\ [a \cdot g]_{q_1} \\ \vdots \\ [a \cdot g]_{q_k} \end{bmatrix} \cong \begin{bmatrix} [a]_{q_0} & [aB]_{q_0} & \cdots & [aB^{\ell-1}]_{q_0} \\ [a]_{q_1} & [aB]_{q_1} & \cdots & [aB^{\ell-1}]_{q_1} \\ \vdots & \vdots & \ddots & \vdots \\ [a]_{q_k} & [aB]_{q_k} & \cdots & [aB^{\ell-1}]_{q_k} \end{bmatrix}$$

↙ NTT

$$\left(a, aB, \dots aB^{\ell-1}\right)_{\mathsf{DCRT}}$$

Note that $\hat{a}$ is a vector of polynomials (the gadget decomposition of a), a is a polynomial, and c_i are the coefficients of a. When we apply the second decomposition,

$a \to \hat{a}$, we get a 3-dimensional matrix of residues $\iff$ a 2-dimensional matrix of polynomial residues $\iff$ a vector of RNS polynomials $\therefore P_{\mathsf{BV}}(a) \leftrightarrow (a, aB, \dots aB^{\ell-1})$.

Then constructing $P_Q(a)$ is in some way equivalent to constructing $\hat{a}$, and the only difference between $P_Q(a)$ and $\hat{a}$ in practice are the indices of the elements as we ignore the 0-towers. In other words we can construct $\hat{a}$ and in the external product skip any computations that involves a 0-tower.

Definition 3.1. RGSW-BV Data Structure We define a data structure representing an RGSW ciphertext as a vector of RLWE ciphertexts. Let $G = I_2 \otimes P_{\mathsf{BV}}(1) = I_2 \otimes P_{\mathsf{HPS}}(g)$ and $G^{-1}(x) = [D_{\mathsf{HPS}}(x_0)|D_{\mathsf{HPS}}(x_1)]$ using the BV gadgets...

$$C = Z + mG = Z + m \begin{pmatrix} P(1) & 0 \\ \vdots & \vdots \\ P(1) & 0 \\ 0 & P(1) \\ \vdots & \vdots \\ 0 & P(1) \end{pmatrix} \tag{3.1}$$

Let $m_0 = m_0 \cdot P(1)$

Algorithm 3.1 Encrypt RGSW-BV

Input: Ring element m in NTT form and decomposition parameter ℓ
Output: RGSW as matrix of $2 \times k\ell$ polynomials

Definition 3.2. RGSW-BV External Product To calculate the external product using the BV decomposition using the RNS basis $\mathcal{B}_Q$ and decomposition parameter ℓ, we

And the external product:

$$x \boxdot Y = G^{-1}(x) \cdot Y \tag{3.2}$$
$$G^{-1}(x)(Z + m_y \cdot G) \tag{3.3}$$
$$G^{-1}(x) \cdot Z + m_y \cdot (G \cdot G^{-1}(x)) \tag{3.4}$$
$$G^{-1}(x) \cdot Z + m_y \cdot x \tag{3.5}$$

For the most optimal result, we flatten the loop to allow effective parallelization of each tower:

Algorithm 3.2 Encrypt RGSW (optimized)

Input: Polynomial m and param ℓ
Output: RGSW as matrix of $2 \times k\ell$ polynomials

Initialize a vector $\mathbf{C}$ of $2 \cdot k \cdot \ell$ RLWE ciphertexts
for $row \in [0, 2 \cdot k \cdot \ell)$ **in parallel do**
    $h \leftarrow \lfloor row/(k\ell) \rfloor$
    $j \leftarrow \lfloor (row \bmod k\ell)/\ell \rfloor$
    $i \leftarrow \lfloor (row \bmod k\ell) \bmod \ell \rfloor$
    $z \leftarrow \mathsf{RLWE}(0) = (c_0, c_1)$
    $c_h[j] \leftarrow [c_h[j] + mB^i]_{q_j}$
    $\mathbf{C}[hk\ell + j\ell + i] = z$
end for
return C

Decomposition

Digit extraction is performed in the coefficient domain. This is the main inefficiency in the HPS/BV-RNS gadget scheme. Let $\check{a} = ([d_0]_B, [d_1]_B, \cdots, [d_{k-1}]_B)$ denote the decomposition of a using **??**. As $D_Q(a) = \left([a]_{q_0}, [a]_{q_1}, \cdots, [a]_{q_i}\right) = a$, the decomposition into $D_Q(\check{a})$ is equivalent to finding $(D_Q(d_0), D_Q(d_1), \cdots, D_Q(d_{k-1}))$.

	Digit 0 (B^0)	**Digit 1** (B^1)	$\cdots$	**Digit** $\ell - 1$ $(B^{\ell-1})$
Tower q_0	$d_{0,0}$	$d_{0,1}$	$\cdots$	$d_{0,\ell-1}$
Tower q_1	$d_{1,0}$	$d_{1,1}$	$\cdots$	$d_{1,\ell-1}$
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
Tower q_{k-1}	$d_{k-1,0}$	$d_{k-1,1}$	$\cdots$	$d_{k-1,\ell-1}$

Table 3.1: The $k \times \ell$ structure of the Double-CRT Decomposition $D_Q(c)$.

Modulo Powers-of-2

The modulo operation is slow because it cannot be done in parallell. If we assume $B = 2^b$ is a power of 2 — which we can enforce by accepting $\log(B) = b$ as the input parameter instead of B — then we can use the fact that right-shift $(a \gg b) = a \bmod 2^b$ or something to make the modulo operation into a bit-wise (and thus easily parallellizable) operation.

Algorithm 3.3 Optimized Branchless Signed LSB Decomposition ($B = 2^b$)

Input: Coefficient $a \in \mathbb{Z}$, bit-width b, number of digits ℓ
Output: Vector of signed digits $\mathbf{d} = (d_0, d_1, \ldots, d_{\ell-1})$ where $d_j \in [-2^{b-1}, 2^{b-1})$

1: $a' \leftarrow a$
2: mask $\leftarrow 2^b - 1$
3: **for** $j = 0$ to $\ell - 1$ **do**
4: $\quad u \leftarrow a' \,\&\, \text{mask}$ ▷ Extract the lowest b bits
5: $\quad \text{carry} \leftarrow u \gg (b-1)$ ▷ Extract the highest bit of u (1 or 0)
6: $\quad d_j \leftarrow u - (\text{carry} \ll b)$ ▷ Branchlessly shift to negative range
7: $\quad a' \leftarrow (a' \gg b) + \text{carry}$ ▷ Shift down and add the carry
8: **end for**
9: **return d**

3.2.2 GHS/Hybrid

- Reuses OpenFHE internals, including Hybrid keyswitching settings
- Downside: parameters tied to keyswitching, can be a problem if a program uses both keyswitching and RGSW

Pre-computed Values

For BV:
- $B^i \bmod q_j$ for all i, j
- Get $B = \lceil \max_j \|q_j\| / \hat{\ell} \rceil$ from ℓ

For GHS/Hybrid:
-

3.3 Implementation Details (sPAR)

- Does not implement the bucket sorting step: negligible impact

3.4 Experimental Setup

- Setup phase (considered "free")

Chapter 4
Results and Discussion

– Parameters used and time per user/message of each step in the protocol
– Differences between BV and GHS gadgets for the external product
– AVX512 improvements

Chapter 5
Conclusion and Future Work

Conclusion

– Achieved better time than expected

Future Work

– BGV SIMD capabilities [**<empty citation>**] not utilized

References

[BEHZ16] J.-C. Bajard, J. Eynard, *et al.*, *A full RNS variant of FV like somewhat homomorphic encryption schemes*, Cryptology ePrint Archive, Paper 2016/510, 2016. [Online]. Available: https://eprint.iacr.org/2016/510.

[BGV11] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, *Fully homomorphic encryption without bootstrapping*, Cryptology ePrint Archive, Paper 2011/277, 2011. [Online]. Available: https://eprint.iacr.org/2011/277.

[CCR19] H. Chen, I. Chillotti, and L. Ren, "Onion ring oram: Efficient constant bandwidth oblivious ram from (leveled) tfhe", in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '19, London, United Kingdom: Association for Computing Machinery, 2019, pp. 345–360. [Online]. Available: https://doi.org/10.1145/3319535.3354226.

[CGGI18] I. Chillotti, N. Gama, *et al.*, *TFHE: Fast fully homomorphic encryption over the torus*, Cryptology ePrint Archive, Paper 2018/421, 2018. [Online]. Available: https://eprint.iacr.org/2018/421.

[Gen09] C. Gentry, "A fully homomorphic encryption scheme", Ph.D. dissertation, Stanford University, 2009. [Online]. Available: https://crypto.stanford.edu/craig.

[GHS12] C. Gentry, S. Halevi, and N. P. Smart, *Homomorphic evaluation of the AES circuit*, Cryptology ePrint Archive, Paper 2012/099, 2012. [Online]. Available: https://eprint.iacr.org/2012/099.

[HPS18] S. Halevi, Y. Polyakov, and V. Shoup, *An improved RNS variant of the BFV homomorphic encryption scheme*, Cryptology ePrint Archive, Paper 2018/117, 2018. [Online]. Available: https://eprint.iacr.org/2018/117.

[HS18] S. Halevi and V. Shoup, *Faster homomorphic linear transformations in HElib*, Cryptology ePrint Archive, Paper 2018/244, 2018. [Online]. Available: https://eprint.iacr.org/2018/244.

[Ko26] R. Ko, *The beginner's textbook for fully homomorphic encryption*, 2026. [Online]. Available: https://arxiv.org/abs/2503.05136.

[KPZ21] A. Kim, Y. Polyakov, and V. Zucca, *Revisiting homomorphic encryption schemes for finite fields*, Cryptology ePrint Archive, Paper 2021/204, 2021. [Online]. Available: https://eprint.iacr.org/2021/204.

[RAD78] R. L. Rivest, L. Adleman, and M. L. Dertouzos, “On data banks and privacy homomorphisms”, in *Foundations of Secure Computation*, 1978, pp. 169–180.

[SEV+15] Y. Sun, A. Edmundson, *et al.*, “RAPTOR: Routing attacks on privacy in tor”, in *24th USENIX Security Symposium (USENIX Security 15)*, Washington, D.C.: USENIX Association, Aug. 2015, pp. 271–286. [Online]. Available: https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/sun.

[SW21] E. Shi and K. Wu, *Non-interactive anonymous router*, Cryptology ePrint Archive, Paper 2021/435, 2021. [Online]. Available: https://eprint.iacr.org/2021/435.

Appendix A
Decomposition

Generally considered *"well-known"* in the literature, but stated here to avoid confusion. Following the definitions from [Ko26], a "*decomposition is a mathematical technique used to represent a number in a smaller base (radix) while preserving its value.*"

Definition A.1. Digit Decomposition A

Definition A.2. Gadget Decomposition B

Definition A.3. Gadget Property C

CRT is a decomposition!

Appendix B
Number Theory

B.1 Chinese Remaineder Theorem

Could be stated here just to be more complete

B.2 Number Theoretic Transform

- Turns a convolution into a nega/cyclic
- Same as FFT but over different ring

Appendix C
RNS Gadgets

C.1 RNS Gadgets

- Establish HPS, BV and GHS/Hybrid gadgets, commonly used in RNS keyswitching
- BV family
  - HPS, closely related to BEHZ
  - BV per limb
- GHS
- Hybrid

C.1.1 Brakerski-Vaikuntanathan

- Digit decomposition is common in keyswitching and other applications, including Gentry-Sahai-Waters (GSW) ciphertexts and TFHE [GHS12; CGGI18; BGV11]
- Used extensively in [**BV**]; decomposition-only key switching called BV-style
- Not directly applicable in RNS without multi-precision integers, but some 'translations' exist

HPS Gadget

- First instantiated in [BEHZ16]
- Uses CRT instead of number decomposition

$$D_Q(a) = \left(\left[a \left(\frac{Q}{q_0} \right)^{-1} \right]_{q_0}, \ldots, \left[a \left(\frac{Q}{q_i} \right)^{-1} \right]_{q_i} \right) \tag{C.1}$$

$$P_Q(a) = \left(\left[a \frac{Q}{q_0} \right]_Q, \ldots, \left[a \frac{Q}{q_i} \right]_Q \right) \tag{C.2}$$

Lemma C.1. *The vectors* (C.6) *and* (C.7) *satisfy the gadget property.*

$$\langle D_Q(a), P_Q(b)\rangle = a \cdot b \pmod{Q} \tag{C.3}$$

- Let $\hat{q}_j = Q/q_j$
- Halevi, Polyakov and Shoup [HPS18] showed that the $\hat{q}_j$ factors can be moved into $P_Q(a)$ without violating Lemma C.3
- This makes the decomposition free for an RNS polynomial, as it simply isolates each tower

$$D_{\mathsf{HPS}}(a) = \left([a]_{q_0}, [a]_{q_1}, \cdots, [a]_{q_i}\right) \tag{C.4}$$

$$P_{\mathsf{HPS}}(a) = \left(a \cdot \left[\hat{q}_0^{-1}\right]_{q_0} \cdot [\hat{q}_0]_Q, \ldots, a \cdot \left[\hat{q}_\ell^{-1}\right]_{q_i} \cdot [\hat{q}_\ell]_Q\right) \tag{C.5}$$

Lemma C.2. *The HPS gadget applied to an RNS (CRT) polynomial is free, and equivalent to isolating each limb.*

$$\textit{Let } e_j = [\hat{q}_j^{-1}]_{q_j} \cdot \hat{q}_j \bmod Q, \textit{ then } e_j \bmod q_i = \begin{cases} 1 & i = j \\ 0 & i \neq j \end{cases}$$

$$\textit{and } P_{\mathsf{HPS}}(a) = (a \cdot e_0, a \cdot e_1, \ldots, a \cdot e_{k-1})$$

Proof.

$$\begin{aligned} i = j \implies e_j &\equiv \hat{q}_j^{-1} \cdot 0 \equiv 0 && \pmod{q_i} \\ i \neq j \implies e_j &\equiv \hat{q}_j^{-1} \cdot \hat{q}_j \equiv 1 && \pmod{q_i} \end{aligned}$$

□

- Note: The e_j factors are static and can be pre-calculated

C.1.2 Brakerski-Vaikuntanathan

- BV [**BV11**] = decomposition-only key switching: no modulus extension, everything stays at modulus Q
- Split ciphertext element into small digits $D(a)$; key encrypts s scaled by matching constants $P(s)$; correctness via gadget property $\langle D(a), P(b)\rangle \equiv a \cdot b \pmod{Q}$ (ref. abstract gadget section)
- Noise governed by digit size: $v_{\text{ks}} = \langle D(c_1), \vec{\mathbf{e}} \rangle$, so $\|v_{\text{ks}}\|_\infty$ scales with $\max_j \|D(c_1)_j\|_\infty$
- Classical instantiation: radix-ω digits of each coefficient, $D_\omega(a) = (a_0, \ldots, a_{\ell_\omega - 1})$ with $a = \sum_m a_m \omega^m$, $P_\omega(b) = (b, b\omega, \ldots, b\omega^{\ell_\omega - 1})$, digits bounded by $\omega/2$
- Transition / the problem: radix decomposition reads off the *positional* representation of a coefficient — in RNS this is never materialized; coefficients exist only as residues mod q_j, and reconstructing the multiprecision value would defeat the purpose of RNS

RNS Instantiation

- Resolution: no reconstruction needed — *the CRT is itself a decomposition*
- Each residue $[a]_{q_j}$ is a word-sized digit; the CRT reconstruction formula

$$a \equiv \sum_j \left[a\,\hat{q}_j^{-1}\right]_{q_j} \cdot \hat{q}_j \pmod{Q}, \qquad \hat{q}_j = Q/q_j$$

  expresses a as a linear combination of digits with fixed constants — exactly the shape of a gadget
- Formalizing yields the RNS gadget, first given in [BEHZ16]:

$$D_Q(a) = \left(\left[a\,\hat{q}_0^{-1}\right]_{q_0}, \ \ldots, \ \left[a\,\hat{q}_\ell^{-1}\right]_{q_\ell}\right) \tag{C.6}$$

$$P_Q(a) = \left([a\,\hat{q}_0]_Q, \ \ldots, \ [a\,\hat{q}_\ell]_Q\right) \tag{C.7}$$

Lemma C.3. Gadget property of the RNS gadget $\langle D_Q(a), P_Q(b)\rangle \equiv a \cdot b \pmod{Q}$.

- Remark (invariance under folding): the lemma is invariant under moving per-coordinate constants between D and P — the j-th summands differ by a multiple of $q_j\hat{q}_j = Q$
- HPS [HPS18] exploit this: move the $\left[\hat{q}_j^{-1}\right]_{q_j}$ factors from D_Q into P_Q:

$$D_{\mathsf{HPS}}(a) = \left([a]_{q_0}, \ [a]_{q_1}, \ \ldots, \ [a]_{q_\ell}\right) \tag{C.8}$$

$$P_{\mathsf{HPS}}(a) = \left(a \cdot \left[\hat{q}_0^{-1}\right]_{q_0} \cdot [\hat{q}_0]_Q, \ \ldots, \ a \cdot \left[\hat{q}_\ell^{-1}\right]_{q_\ell} \cdot [\hat{q}_\ell]_Q\right) = (a \cdot e_0, \ \ldots, \ a \cdot e_\ell) \tag{C.9}$$

- $D_{\mathsf{HPS}}(a)$ is exactly the RNS representation of a: decomposition is *free* for an RNS polynomial — limb lookup, zero arithmetic
- The $e_j = \left[\hat{q}_j^{-1}\right]_{q_j} \cdot \hat{q}_j \bmod Q$ are the CRT idempotents (textbook CRT, same objects as in the proof of Lemma C.3); static, precomputed into the key at keygen
- Noise unchanged by folding: digit bound $q_j/2$ in both forms
- Closing / forward pointer: digits bounded by $\tilde{q}/2$ with $\tilde{q} = \max_j q_j$, so noise scales with limb size; when smaller digits are needed, each limb — now word-sized — can itself be radix-decomposed (next section). Note: the CRT gadget is what makes radix decomposition applicable again

Per-Limb Digit Decomposition

- Goal: reduce digit size from $\tilde{q}/2$ to $\omega/2$ for radix $\omega < \tilde{q}$

– Key fact: radix-ω decomposition of a digit is an *exact integer identity*, $d_j = \sum_m \omega^m d_{j,m}$ over $\mathbb{Z}$ — so it composes with *any* gadget whose digits are small integer representatives

Lemma C.4. Gadget composition *Let (D, P) satisfy the gadget property with integer digit representatives $|D(a)_j| \le q_j/2$. Then*

$$D'(a) = (d_{j,m})_{j,m}, \qquad \text{where } D(a)_j = \sum_m \omega^m d_{j,m},\ |d_{j,m}| \le \omega/2$$

$$P'(b) = (\omega^m \cdot P(b)_j)_{j,m}$$

satisfies $\langle D'(a), P'(b)\rangle \equiv a \cdot b \pmod{Q}$.

– Instantiation with the RNS gadget (HPS form): $D'(a)$ = radix decomposition of each limb of the RNS representation

$$D'(a) = \left(\left([a]_{q_j}\right)_m\right)_{j,m}, \qquad P'(b) = (b \cdot \omega^m e_j)_{j,m}$$

– Still no scalar multiplications on the ciphertext side: all constants (e_j, ω^m) live in P', precomputable; ciphertext side is limb lookup + word-sized radix split
– Cost/noise tradeoff: $\ell_{\omega,\tilde{q}} = \lfloor \log_\omega \tilde{q} \rfloor + 1$ times more gadget components (key size, NTTs), digits shrink $\tilde{q}/2 \to \omega/2$; noise bound as in [KPZ21], App. B.2.1 (second level of decomposition)
– Naming remark: no modulus extension is performed, so this remains a BV-type key switch; we refer to it simply as *BV* in the remainder of this thesis (and in the implementation)
– Remark (forward pointer): Lemma C.4 applies to any gadget with small integer digits — in particular to the digits of hybrid key switching (Section **??**)

Appendix D
Implementation

D.1 Brakerski-Vaikuntanathan

As established, digit-decomposing each RNS prime with ℓ digits is equivalent to digit-decomposing the full polynomial with $k \cdot \ell$ digits, where $Q = \prod_{i=0}^{k-1} q_i$ or the basis $\mathcal{B}_Q$.

D.1.1 Powers of Base

1. $a \to (a, aB, \ldots, B^{\ell-1})$
2. $g = (1, B, \cdots, B^{\ell}, B, \ldots, B, B^2, \ldots, B^{\ell-1}, \ldots B^{\ell-1})$??

Algorithm D.1 Powers of Base (Optimized)

Input: RNS coefficients $x = ([x]_{q_0}, ..., [x]_{q_{k-1}})$, base $B = 2^b$, number of digits ℓ
Output: ℓ RNS/CRT polynomials $(a, aB, \ldots, B^{\ell-1})$

```
Initialize a vector x' of size ℓ
for i ∈ [0, ℓ) in parallel do
    for j ∈ [0, k) do
        x'_i[j] := x[j] · B^i mod q_j
    end for
end for
return x'
```

- B (or ℓ) is implicit from $\ell = \lfloor \log_B (|\max_i q_i|) \rfloor + 1$, and since $B = 2^b$ is a power of 2 we get $\ell = \lfloor \log(\max_i q_i)/b) \rfloor + 1$.
- B^i or $B^i \bmod q_j$ can be calculated ahead of time.
- The for loop can be flattened to $0 \leq idx < k \cdot \ell$ to allow more parallelization.
- For maximum efficiency, the function can be encoded into the encryption function.
- The fully optimized encryption functions becomes:

Algorithm D.2 Encrypt BV-RGSW (Optimized)

Input: Plaintext $m = ([m]_{q_0}, ..., [m]_{q_{k-1}})$, base $B = 2^b$, number of digits ℓ
Output: $\mathsf{RGSW}(m)$ with $2k\ell$ rows

1: Initialize a vector C of $2k\ell$ `DCRTPoly` objects (c_0, c_1)
2: Set m to RNS-coefficients mode
3: **for** $idx \in [0, \ell \cdot k)$ **in parallel do**
4: $\quad (i, j) \leftarrow [\lfloor idx/\ell \rfloor,\ idx \bmod \ell]$
5: $\quad mg \leftarrow m[j] \cdot B^i \bmod q_j$ ▷ Retrieve $B^i \bmod q_j$ from lookup table
6: $\quad C_{idx} \leftarrow \mathsf{HE.Encrypt}(0) + (\mathsf{NTT}(mg), 0)$
7: $\quad C_{idx+\ell} \leftarrow \mathsf{HE.Encrypt}(0) + (0, \mathsf{NTT}(mg))$
8: **end for**
9: **return** C

D.1.2 Decompose

1.

Algorithm D.3 BV Decompose (Optimized)

Input: Coefficients $x = ([x]_{q_0}, ..., [x]_{q_{k-1}})$, base $B = 2^b$, number of digits ℓ
Output: ℓ polynomials of signed digits $(d_0, d_1, \ldots, d_{\ell-1})$ where $d_j \in [-B/2, B/2)$

1: $a' \leftarrow a$
2: **for** xxxx **do**
3: $\quad$ xxxx
4: **end for**
5: **return** $\{d\}$

– Optimized/folded form:

Algorithm D.4 External Product $x \boxdot Y$ (Optimized BV-RGSW)

Input: RLWE ciphertext $x = (x_0, x_1) \in R_q^2$, RGSW ciphertext $Y = \mathsf{RGSW}(m)$ with $2k\ell$ rows, base $B = 2^b$, decomposition parameter/digits ℓ
Output: RLWE ciphertext encrypting $m \cdot m_x$

$\mathsf{acc} \leftarrow (0, 0)$ ▷ accumulator in evaluation (NTT) form
for $c \in \{0, 1\}$ **do** ▷ decompose each ciphertext component
  for $idx \in [0, \ell \cdot k)$ **in parallel do**
    $(i, j) \leftarrow [\lfloor idx/\ell \rfloor,\ idx \bmod \ell]$
    $r \leftarrow [x_c]_{q_j}$ ▷ j-th CRT residue, coefficient form
    $d_{idx} \leftarrow \mathsf{Digit}_B(r, i)$ ▷ i-th base-B digit; coeffs in $[0, B)$ or $(-B/2, B/2]$
    $\tilde{d}_{idx} \leftarrow \mathsf{CRTExtend}(d_{idx})$ ▷ lift small digit poly to all k moduli, then NTT
  end for
  $\mathsf{acc} \leftarrow \mathsf{acc} + \sum_{idx} \tilde{d}_{idx} \cdot Y_{idx+c\cdot k\ell}$ ▷ 2 poly mults per row
end for
return acc